



Flutter

Zero to One

Dart의 기초

새로운 언어를 배울 때

- ✓ 이전에 배웠던 언어들과 비교해보기

<https://dart.dev/resources/coming-from/js-to-dart>

- ✓ 내가 만들었던 예전 코드들 번역해보기

그럼에도 불구하고

Dart에는 익숙한 문법들이 많습니다

변수 vs 상수

변수 만들기

```
var name = '홍길동';
```

```
int myAge = 25;
```

```
myAge = 35;
```

✓ var를 써주거나 타입을 명시

✓ 다른 값 재할당 가능

상수 만들기

```
final String name = '홍길동';
```

```
final int age = 25;
```

```
age = 35; ❌
```

✅ final를 앞에 써주기

✅ 앱이 실행된 후 초기화 됨

⚠️ 다른 값 재할당 불가능

상수 만들기

```
const double PI = 3.14;
```

- ✓ const를 앞에 써주기
- ✓ 앱이 실행되기 전에 이미 초기화 됨
- ⚠ 다른 값 재할당 불가능

변수와 상수

- ✓ var: 변경 가능한 변수
- ✓ final: 앱 실행 후 할당됨. 변경 불가
- ✓ const: 앱 실행 전 할당됨. 변경 불가

데이터 타입

숫자 타입

```
int age = 25;
```

```
double height = 178.5;
```

숫자 타입

```
int age = 25;
```

```
age += 1;
```

```
age++;
```

문자 타입

```
String name = "홍길동";
```

문자 타입

```
String name = "홍길동";  
print("안녕하세요, ${name}님");
```

✓ 문자열 안에 \$나 \${}를 넣으면
변수를 넣을 삽입할 수 있다.

문자 타입

```
String name = "홍길동";  
print("안녕하세요," + name + "님");
```

✓ 문자열끼리 **+**로 연결할 수 있다.

문자 타입

QUIZ. 문자열 안에 따옴표를 넣으려면?

"그는 말했다 "안녕" "

숫자 → 문자

```
int age = 42;  
String string = age.toString();
```

✓ toString을 사용하면 숫자를 문자로 바꿀 수 있다.

문자 → 숫자

```
String age = "42";  
int num = int.parse(age);
```

- ✅ `parse`를 사용하면 문자를 숫자로 바꿀 수 있다.
단, 엉뚱한 문자를 넣으면 에러 발생!

Yes or No

```
bool 출근하고싶니 = false;
```

```
출근하고싶니 = true;
```

```
bool 퇴근하고싶니 = !출근하고싶니;
```



dynamic

```
dynamic someValue = true;  
someValue = 100;  
someValue = "바둑이";  
someValue = 178.0;
```

Nullable

```
String? 월급 = "있었는데요";  
        월급 = null;    // 없었습니다.
```

- ✓ 빈 값(null)이 될 가능성이 있는 타입
- ✓ null인지 아닌지 항상 체크하고 써야 함

Nullable

`String? 월급;`

`월급.퍼가요();` ❌

`월급?.퍼가요();` ✅

✅ Nullable 변수는 그냥 참조할 수는 없습니다.

✅ Nullable 변수를 **주어로 삼을 때에는 ?**를 붙여줍니다.

만약 null이면 실행되지 않음

Nullable

```
int? nullable = null;
```

```
int nonNullable = null; ❌
```

✅ nullable이 아닐 때는 null이 들어갈 수 없다!

Nullable

```
int? 월급;
```

```
int 통장 = 월급 ?? 0;
```

- ✓ Nullable이 null 일 때 다른 값으로 대체하고 싶으면 ?? 를 사용합니다.

리스트 & 맵

리스트

```
List<String> 세계1주 = [  
    '김동준', '김수민', '이성혁',  
    '전경서', '정수임', '조소현',  
];
```

✓ 순서가 있는 집합

리스트

```
List<String> 세계1주 = ['김동준', '김수민', '이성혁'];
```

```
세계1주[2] = '전경서';
```

✓ 순서가 있는 집합

리스트

```
List<String> 세계1주 = ['전경서', '김수민'];
```

```
세계1주.add( '이성혁' );           // 추가하기
```

```
세계1주.remove( '김동준' );        // 삭제하기
```

```
세계1주.sort();                     // 정렬하기
```

리스트 예쁘게 쓰기

```
List<String> 세계1주 = [  
    '김동준', '김수민', '이성혁',  
    '전경서', '정수임', '조소현',  
];
```

- ✅ 괄호가 닫히기 전에 꼭 ,를 넣어줍니다!
그러면 자동으로 예쁘게 코드가 정렬됩니다!

맵

```
Map<String, int> scores = {  
    '국어': 90, '영어': 85,  
};
```

✓ 키-값 쌍의 집합 (값에 이름을 붙임)

맵

```
Map<String, int> scores = {  
    '국어': 90, '영어': 85,  
};
```

```
scores['수학'] = 95;
```

```
print(scores['수학']);
```

맵

```
Map<String, int> scores = {  
    '국어': 90, '영어': 85,  
};
```

```
print(scores['과학']); // null
```

```
scores.containsKey('과학'); // false
```

✅ 키가 있는지 항상 먼저 체크하는 습관

맵

```
Map<String, int> scores = {  
    '국어': 90, '영어': 85,  
};
```

```
final science = scores['과학'];
```

QUIZ. 여기서 **science**의 데이터 타입은?

✅ 키가 있는지 항상 먼저 체크하는 습관

맵 예쁘게 쓰기

```
Map<String, int> scores = {  
    '국어': 90,  
    '영어': 85,  
    '수학': 100,  
};
```

- ✅ 괄호가 닫히기 전에 꼭 ,를 넣어줍니다!
그러면 자동으로 예쁘게 코드가 정렬됩니다!

제네릭

```
List<String> fruits = {'사과'};  
Map<String, int> scores = {'국어': 90};
```

✓ 어떤 타입을 ‘편식’할 것인지 명시하는 것

반복문

for

```
for (초기식; 조건식; 증감식) {  
    // 반복 실행 코드  
}
```

for

```
for (int i = 0; i < 5; i++) {  
    print('반복 횟수: $i');  
}
```

for-in

```
for (var fruits in fruits) {  
    // 반복 실행 코드  
}
```

리스트 안에서 **for** 사용하기

```
[  
    for (String name in r3Play)  
        Text(name),  
]
```

- ✓ 리스트 안에서 **for**문의 제목을 쓰면
리스트 안에 데이터를 반복해서 채워넣을 수 있습니다.
- ✓ **for**문의 종괄호 **}**는 생략합니다.

while

```
int count = 0;

while (count < 5) {
    print('카운트: $count');
    count++;
}
```

조 건 문

AND

```
int age = 20;  
bool isStudent = true;  
  
if (age >= 18 && isStudent) {  
    print( '대학생 입니다. ' );  
}
```

OR

```
int age = 20;  
bool isStudent = true;  
  
if (age < 18 || isStudent) {  
    print( '미성년자이거나 학생입니다.' );  
}
```

if else

```
if (조건) {  
    // 조건이 참일 때 실행  
}  
else if (다른 조건) {  
    // 다른 조건이 참일 때 실행  
}  
else {  
    // 모든 조건이 거짓일 때 실행  
}
```

리스트 안에서 if 사용하기

```
[  
    if (name.isNotEmpty)  
        Text(name),  
]
```

- ✓ 리스트 안에서 if문의 제목을 쓰면
조건을 충족할 때만 데이터를 채워넣을 수 있습니다.
- ✓ if문의 종괄호 {}는 생략합니다.

switch

```
var grade = 'A';

switch (grade) {
  case 'A':
    print('우수');
    break;
  case 'B':
    print(' 좋음 ');
    break;
  default:
    print('노력 필요');
}
```

삼항 연산자

조건 ? 참일 때 값 : 거짓일 때 값;

```
String result = (age >= 18) ? '성인' : '미성년자';
```

실습



장바구니 총합 계산 🛒

배달 앱의 장바구니에 담긴 음식들의
가격 총합을 계산하는 프로그램을 만들어봅시다.

각 음식의 이름과 가격은 **Map**에 저장되어 있습니다.

장바구니 총합 계산 🛒

1. 장바구니에 담긴 상품 맵 만들기 (상품명-가격)

```
Map<String, int> foods = {  
    '김치찌개': 9000, '비빔밥': 12000, '단무지': 500, '마라탕': 15000  
};
```

2. 가격의 총합을 계산하여 아래와 같이 출력

ex) "장바구니의 총 가격은 ♦♦♦♦원 입니다."

랜덤 저녁 메뉴 출력 🍴

1. 점심 메뉴 리스트 만들기

```
List<String> menu = ['김치찌개', '비빔밥', '돈까스'];
```

2. 랜덤으로 하나 선택하여 출력

3. 힌트

```
final random = Random();
```

함수

함수 선언

```
반환타입 함수이름(파라미터) {  
    // 실행 코드  
    return 값;  
}
```

함수 선언

```
int add(int a, int b) {  
    return a + b;  
}
```

함수 줄여쓰기

```
int add(int a, int b) => a + b
```

✓ 함수가 한 줄 짜리일 때, 줄여 쓸 수 있다!

함수 줄여쓰기

```
int add(int a, int b) => a + b  
    add(1, 3)
```

✓ 함수가 한 줄 짜리일 때, 줄여 쓸 수 있다!

위치 파라미터

```
int add(int a, int b) => a + b  
      add(1, 3)
```

- ✓ 위치(순서)에 의해 구분되는 파라미터
- ✓ 선언된 순서에 따라 인자를 넣어준다
- ✓ 위치 파라미터는 항상 맨 앞에 선언

이름 있는 파라미터

```
int add({int a, int b}) => a + b
```

```
add(a: 1, b: 3)
```

```
add(b: 3, a: 1)
```

- ✓ 이름에 의해 구분되는 파라미터
- ✓ 이름에 맞게 인자를 넣어준다
- ✓ 항상 위치 파라미터 뒤에 선언해줄 것

이름 있는 파라미터

```
int add({required int a, required int b}) => a + b
```

```
add(a: 1, b: 3)
```

```
add(b: 3, a: 1)
```

- ✓ 이름을 깜빡할 수 있으니 **required**를 넣어줘야 한다
- ✓ **required**가 들어갈 필요가 없는 경우?

이름 있는 파라미터

```
int add({int? a, int? b}) => a! + b!
```

```
add(a: 1)
```

```
add(b: 3)
```

✓ 이름을 깜빡할 수 있으니 **required**를 넣어준다

✓ **required**가 들어갈 필요가 없는 경우?

타입이 nullable일 때!

함수 호출 예쁘게 하기

```
add(  
    a: 1,  
    b: 2,  
);
```

- ✅ 괄호가 닫히기 전에 꼭 ,를 넣어줍니다!
그러면 자동으로 예쁘게 코드가 정렬됩니다!

실습



학생부 정리하기 📖

학생들의 이름과 성적이 담긴 리스트를
이름 및 성적으로 검색할 수 있는 함수를 만들어봅시다

학생부 정리하기 📖

1. 아래 리스트를 복사해서 main 함수 위에 붙여넣으세요

```
const List<Map<String, dynamic>> students = [

  {"name": "김민준", "score": 85}, {"name": "이서윤", "score": 92}, {"name": "박지우", "score": 78}, {"name": "최예은", "score": 89}, {"name": "정하윤", "score": 91}, {"name": "강서연",
"score": 74}, {"name": "윤도윤", "score": 88}, {"name": "조현우", "score": 95}, {"name": "오하은", "score": 77}, {"name": "신우진", "score": 83}, {"name": "홍서준", "score": 82}, {"name":
"임주원", "score": 94}, {"name": "류채은", "score": 72}, {"name": "김하람", "score": 86}, {"name": "문예준", "score": 80}, {"name": "최지호", "score": 90}, {"name": "박다인", "score": 79},
{"name": "이준우", "score": 88}, {"name": "김유찬", "score": 87}, {"name": "윤태민", "score": 93}, {"name": "전도현", "score": 81}, {"name": "서연우", "score": 92}, {"name": "한수민",
"score": 76}, {"name": "유시현", "score": 89}, {"name": "최지안", "score": 84}, {"name": "신예지", "score": 85}, {"name": "장하준", "score": 74}, {"name": "정세준", "score": 78}, {"name":
"송윤아", "score": 90}, {"name": "양지호", "score": 87}, {"name": "백다은", "score": 82}, {"name": "하지후", "score": 88}, {"name": "강민재", "score": 93}, {"name": "정유진", "score": 75},
{"name": "이재윤", "score": 91}, {"name": "박민혁", "score": 79}, {"name": "김태윤", "score": 80}, {"name": "이도현", "score": 84}, {"name": "오주원", "score": 77}, {"name": "조아인",
"score": 92}, {"name": "송다인", "score": 86}, {"name": "허예림", "score": 83}, {"name": "최한별", "score": 89}, {"name": "황지윤", "score": 88}, {"name": "정동현", "score": 90}, {"name":
"김시우", "score": 85}, {"name": "신정우", "score": 93}, {"name": "권다원", "score": 82}, {"name": "임태양", "score": 80}, {"name": "류가람", "score": 87}, {"name": "윤채원", "score": 84},
{"name": "박하준", "score": 91}, {"name": "이은성", "score": 77}, {"name": "최다은", "score": 85}, {"name": "박서준", "score": 90}, {"name": "김지민", "score": 92}, {"name": "장예린",
"score": 76}, {"name": "정태현", "score": 79}, {"name": "신도현", "score": 88}, {"name": "박윤서", "score": 89}, {"name": "윤승현", "score": 81}, {"name": "조민서", "score": 85}, {"name":
"강지민", "score": 78}, {"name": "문서준", "score": 84}, {"name": "양현우", "score": 90}, {"name": "임시연", "score": 87}, {"name": "김다연", "score": 92}, {"name": "최민서", "score": 80},
{"name": "박채윤", "score": 77}, {"name": "이정민", "score": 83}, {"name": "김수현", "score": 75}, {"name": "유다은", "score": 79}, {"name": "조서연", "score": 93}, {"name": "하지우",
"score": 88}, {"name": "장도윤", "score": 84}, {"name": "황유진", "score": 90}, {"name": "이하율", "score": 76}, {"name": "강은서", "score": 81}, {"name": "이서현", "score": 85}, {"name":
"정예린", "score": 89}, {"name": "박도현", "score": 78}, {"name": "김다인", "score": 94}, {"name": "신하윤", "score": 91}, {"name": "문하준", "score": 83}, {"name": "윤하진", "score": 75},
{"name": "전수빈", "score": 88}, {"name": "유서연", "score": 90}, {"name": "이윤서", "score": 77}, {"name": "권민준", "score": 86}, {"name": "조예림", "score": 79}, {"name": "김현우",
"score": 87}, {"name": "윤수아", "score": 92}, {"name": "하서준", "score": 91}, {"name": "이수민", "score": 80}, {"name": "최서현", "score": 78}, {"name": "장준우", "score": 83}, {"name":
"박현우", "score": 82}, {"name": "김지후", "score": 94}, {"name": "송민준", "score": 85}, {"name": "윤가현", "score": 90}, {"name": "김기범", "score": 100}, {"name": "김기범", "score": 70}
```

학생부 정리하기 📖

2. 다음 두 함수를 완성해보세요

```
List searchGradeByName({required String name})
```

```
List searchNameByGrade({required int grade})
```

```
searchGradeByName("김기범"); // ["100", "70"]
```

```
searchNameByGrade(100); // ["김기범"]
```

질문 시간

다음 시간에는
객체지향에 대해 배워보아요 🧐